Taller Práctico de Repaso y Consolidación en C++: Recursividad, Búsqueda Secuencial y Punteros con Método 5R

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Se consolidan tres pilares de bajo nivel en una suite unificada: (a) recursividad con caso base para evitar desbordamiento de pila, (b) búsqueda secuencial O(n) en arreglo contiguo con interrupción mediante *break* y verificación de contigüidad con el operador de dirección, y (c) intercambio físico con punteros y variable temporal. La metodología 5R (Registrar, Reducir, Responder, Reflexionar y Revisar) guía el aprendizaje y la documentación en *CHANGELOG.md*. Las pruebas con suma de 1 a 5 igual a 15, búsqueda de 18 en índice 6 e intercambio de 100 y 500 validan la implementación. Se incluye la estrategia de doble remoto en GitHub y GitLab como resguardo ante caídas para el Proyecto Integrador Anual.

Palabras clave: recursividad, caso base, desbordamiento de pila, búsqueda secuencial, O(n), break, punteros, intercambio, método 5R

Abstract

Three low-level pillars are consolidated in a unified suite: (a) recursion with a base case to prevent stack overflow, (b) O(n) sequential search in a contiguous array with a *break* and contiguity verification via the address-of operator, and (c) physical swapping with pointers and a temporary variable. The 5R methodology (Record, Reduce, Respond, Reflect, and Review) guides learning and documentation in *CHANGELOG.md*. Tests with sum 1 to 5 equal to 15, search for 18 at index 6, and swap of 100 and 500 validate the implementation. A dual-remote strategy on GitHub and GitLab is included as a safeguard for the Annual Integrative Project.

Keywords: recursion, base case, stack overflow, sequential search, O(n), break, pointers, swap, 5R method

Introducción

La primera mitad del ciclo abordó recursividad, arreglos y punteros de forma separada. La Actividad 6 los integra en un solo ejecutable para exigir al estudiante el cambio de modelo mental entre pila, montículo y aritmética de punteros en la misma sesión.

El método 5R estructura el trabajo: Registrar (compilar y observar hexadecimal), Reducir (identificar el veinte por ciento clave: *if* de caso base, *break* y variable temporal), Responder (cuestionario), Reflexionar (aplicar al Proyecto Integrador Anual) y Revisar (envío y registro en *CHANGELOG*).

Revisión de la Literatura

La recursividad es la función que se invoca a sí misma dividiendo el problema; requiere caso base. El caso base es la condición que corta la cadena; sin él se produce recursión infinita. La pila es la región de memoria con disciplina último en entrar, primero en salir para variables locales y direcciones de retorno; cada llamada apila un marco. La búsqueda secuencial O(n) recorre índice por índice; la interrupción con *break* evita iteraciones innecesarias tras el hallazgo. El arreglo contiguo almacena elementos en direcciones consecutivas, con incremento de cuatro bytes por entero (Cormen et al., 2009; Stroustrup, 2013).

Reto	Núcleo (20%)	Riesgo si se omite
Recursividad	if (n <= 0) return 0	Desbordamiento de pila
Búsqueda	break tras hallar	Recorrido completo O(n) innecesario
Intercambio	int temp = *pA	Pérdida de dato sin copia temporal

Nota. Núcleos críticos del taller.

Método

La suite en *src/main.cpp* declara los prototipos *sumaRecursiva* e *intercambiarValores*. El Reto 1 con *sumaRecursiva(5)* produce 15 con traza 5 + 4 + 3 + 2 + 1. El Reto 2 emplea *vectorDatos[10]* contiguo, búsqueda de 18 con ciclo *for* e interrupción,

y muestra la dirección hexadecimal de *&vectorDatos[i]*. El Reto 3 intercambia valores con *intercambiarValores(&x, &y)* y variable temporal.

La compilación se ejecutó con *g++ src/main.cpp -o src/repasogeneral.exe*. Las capturas incluyen la ejecución completa y la traza de memoria. El archivo *CHANGELOG.md* sigue el versionado SemVer con versión *v1.0.0 Baseline Aprobado* y tabla de revisiones DocVer. El doble remoto se configuró con *git remote set-url --add --push* para envíos simultáneos a GitHub y GitLab.

Resultados

La suma de 1 a 5 resultó 15. La búsqueda halló el valor 18 en el índice 6 con dirección hexadecimal e interrumpió con *break* sin recorrer hasta el índice 9. El intercambio invirtió correctamente 100 y 500. La contigüidad se verificó imprimiendo las direcciones en ciclo: las direcciones crecieron de cuatro en cuatro bytes.

Figura 1. Ejecución de los tres retos (capturas/ejecucion_repasogeneral.png). Figura 2. Traza de pila, arreglo y punteros (capturas/traza_memoria.png).

Discusión

Sin caso base, cada llamada apila un marco hasta agotar la pila y provocar desbordamiento y terminación abrupta. La instrucción *if (n <= 0)* constituye el fusible. La interrupción con *break* optimiza el uso del procesador al evitar comparaciones adicionales; en arreglos de miles de sensores del Proyecto Integrador Anual el ahorro es significativo.

La variable temporal es indispensable: sin ella, la secuencia **pA = *pB; *pB = *pA* perdería el valor original de **pA*; la variable temporal lo resguarda. La evidencia de contigüidad es el incremento constante de *sizeof(int)* entre direcciones sucesivas.

Conclusión

Los tres mecanismos quedaron integrados y documentados. El estudiante identifica el reto de mayor complejidad y propone aplicación al Proyecto Integrador Anual, por ejemplo, lecturas de sensores con búsqueda de umbral. Como trabajo futuro se propone

medir tiempos con y sin *break* y comparar recursión con iteración para la suma.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

GeeksforGeeks. (2024). *Recursion in C++ & Linear search*. https://www.geeksforgeeks.org/

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.